



Universidade do Minho

Escola de Engenharia



AutoRobot: Robô Autônomo para Desvio de Obstáculos com Filtro de Kalman

LICENCIATURA EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA

29/12/2024

Trabalho realizado pelo grupo 5:

-  Luís Pedro Cerqueira Baptista a105419
-  Bernardo Salgado a103664
-  Luís Marques a103674



Resumo

Objetivo do trabalho:

O objetivo é criar um robô que, ao detectar obstáculos, seja capaz de se desviar deles de forma eficiente. O sensor ultrassônico fornece leituras da distância dos obstáculos, e o filtro de Kalman é aplicado para garantir que as leituras sejam estáveis e precisas, melhorando a resposta do robô.

Principais Resultados Alcançados:

O robô demonstrou capacidade de identificar e de se desviar de obstáculos em tempo real, com movimentos precisos e sem colisões. O uso do filtro de Kalman reduziu significativamente os ruídos nas leituras do sensor ultrassônico, aumentando a confiabilidade do sistema. Além disso, foi possível otimizar o consumo de energia do robô, prolongando o seu tempo de operação.

Conclusão geral:

O projeto mostrou que a aplicação do filtro de Kalman em sistemas robóticos é eficaz para estabilizar leituras de sensores em ambientes dinâmicos. A abordagem proposta permitiu que o robô reagisse de forma eficiente a obstáculos, demonstrando um comportamento adaptativo e robusto.

O trabalho também destaca a importância de integrar técnicas de processamento de sinal com soluções de hardware para alcançar melhores resultados em robótica.



INTRODUÇÃO

Contextualização:

A procura por robôs autónomos que operem em ambientes dinâmicos motiva o desenvolvimento de sistemas mais complexos e precisos para a navegação e desvio de obstáculos. A confiabilidade das leituras dos sensores é um aspeto crítico para garantir o sucesso desses sistemas.

Este projeto surgiu da necessidade de criar um robô capaz de se desviar eficientemente de obstáculos utilizando sensores ultrassônicos, cuja precisão é melhorada com a aplicação do filtro de Kalman.

O filtro de Kalman destaca-se como uma técnica eficaz para lidar com ruídos e incertezas nas medições, sendo ideal para ambientes reais, onde as condições podem variar constantemente.

Assim, este trabalho procura integrar um sistema simples e eficiente, utilizando o Arduino Mega, um sensor ultrassônico HC-SR04 e motores controlados por um driver L298N.

Objetivos SMART:

- **Specific:** Desenvolver um robô autónomo que utilize o filtro de Kalman para processar leituras do sensor ultrassônico e desviar-se de obstáculos.
- **Measurable:** Garantir que o robô se desvie de pelo menos 95% dos obstáculos em testes realizados em ambientes com obstáculos variados.
- **Attainable:** Utilizar tecnologias acessíveis, como o Arduino Mega e o sensor HC-SR04.
- **Realistic:** Contribuir para a área de robótica autónoma com uma solução que integre controlo de movimento e processamento de sinais em tempo real.
- **Time-bound:** Acabar o projeto em 9 semanas.

Proposta do Projeto

Descrição da ideia principal:

A proposta do projeto foi desenvolver um mini robô autónomo capaz de identificar e desviar-se de obstáculos utilizando 3 sensores ultrassónicos. A aplicação do filtro de Kalman foi fundamental para filtrar ruídos e melhorar a precisão das leituras.

Microcontrolador e tecnologias:

O microcontrolador escolhido foi o Arduino Mega, devido à sua compatibilidade com o sensor ultrassónico HC-SR04 e a facilidade de programação. Outras tecnologias utilizadas incluem o driver de motor L298N e bibliotecas específicas para controlo dos motores e implementação do filtro de Kalman.

Aplicações potenciais:

O robô pode ser aplicado em ambientes industriais para transporte de materiais, em robôs de serviço doméstico para navegação autónoma ou em protótipos educacionais para o ensino de conceitos de robótica e processamento de sinal.



Design do Sistema

Especificações do sistema:

- **Entrada:** Leituras do sensor ultrassônico HC-SR04 (sinais de tempo para calcular distância).
- **Processamento:** Filtro de Kalman para suavizar as leituras do sensor, eliminando ruídos e flutuações.
- **Saída:** Controlo dos motores para ajustar a direção do robô e desviar-se dos obstáculos.

Arquitetura do sistema:

Hardware:

- Sensor ultrassônico HC-SR04 para leitura de distâncias.
- Motores DC ligados ao driver L298N para controlar a movimentação.
- Arduino Mega para processamento e controlo geral.

Software:

- Código em Arduino IDE com algoritmos para filtragem de dados (Kalman) e controlo de motores.

Fluxograma:

1. Ler os dados do sensor ultrassônico.
2. Processar os dados com o filtro de Kalman.
3. Verificar a presença de obstáculos.
4. Ajustar direção e movimentação do robô.
5. Repetir o ciclo.

Desenvolvimento e Implementação

Configuração do ambiente:

O desenvolvimento foi realizado no Arduino IDE. O hardware foi montado utilizando um chassis, sensores, motores e jumpers conectados ao Arduino Mega.

Implementação do filtro de Kalman:

O filtro foi implementado no código do Arduino, utilizando uma estrutura que prevê e atualiza as leituras com base nas medições do sensor e no modelo de incerteza do sistema.

Código:

- Configuração do sensor ultrassônico (Trigger e Echo).

```
// Configuração dos pinos dos sensores ultrassônicos
int trigPinMiddle = A15;    // Trig pin do sensor central
int echoPinMiddle = A14;    // Echo pin do sensor central
int trigPinLeft = A13;      // Trig pin do sensor esquerdo
int echoPinLeft = A12;      // Echo pin do sensor esquerdo
int trigPinRight = A11;     // Trig pin do sensor direito
int echoPinRight = A10;     // Echo pin do sensor direito

void setup() {
  // Configuração dos pinos como OUTPUT ou INPUT
  pinMode(trigPinMiddle, OUTPUT);
  pinMode(echoPinMiddle, INPUT);
  pinMode(trigPinLeft, OUTPUT);
  pinMode(echoPinLeft, INPUT);
  pinMode(trigPinRight, OUTPUT);
  pinMode(echoPinRight, INPUT);
}

// Função para medir a distância usando um sensor ultrassônico
long getDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); // Envia pulso
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH); // Mede duração do pulso
  return duration / 58.2; // Converte para cm
}
```

- Implementação do filtro de Kalman

```
// Variáveis para estado e incerteza
float xMiddle = 0.0, xLeft = 0.0, xRight = 0.0; // Estimativa inicial
float velocity = 36.76; // Velocidade inicial assumida
float P[2][2] = {{200.0, 0}, {0, 50.0}}; // Covariância inicial
float Q[2][2] = {{3, 0}, {0, 1.0}}; // Modelo interno do sistema
float R = 2; // Ruído de medição
float dt = 0.1; // Passo de tempo

void updateEKF(float z, float &x, float &v, float &filteredDistance) {
    // Predição
    float x_pred = x + v * dt;
    float P_pred[2][2];
    P_pred[0][0] = P[0][0] + Q[0][0];
    P_pred[1][1] = P[1][1] + Q[1][1];

    // Atualização
    float y = z - x_pred; // Resíduo
    float S = P_pred[0][0] + R; // Covariância do resíduo
    float K[2]; // Ganho de Kalman
    K[0] = P_pred[0][0] / S;
    K[1] = P_pred[1][0] / S;

    // Atualiza estado
    x = x_pred + K[0] * y;
    v = v; // Velocidade constante
    P[0][0] = (1 - K[0]) * P_pred[0][0];
    P[1][1] = P_pred[1][1];

    filteredDistance = x; // Distância suavizada
}
```

- Controlo dos motores para o desvio de obstáculos.

```
void turnLeft() {
  digitalWrite(fwdright7, HIGH);
  digitalWrite(revright6, LOW);
  digitalWrite(fwdleft5, LOW);
  digitalWrite(revleft4, LOW);
  analogWrite(controlRightMotorSpeed, baseSpeed);
  analogWrite(controlLeftMotorSpeed, 0);
}

void turnRight() {
  digitalWrite(fwdright7, LOW);
  digitalWrite(revright6, LOW);
  digitalWrite(fwdleft5, HIGH);
  digitalWrite(revleft4, LOW);
  analogWrite(controlRightMotorSpeed, 0);
  analogWrite(controlLeftMotorSpeed, baseSpeed);
}

// lógica de desvio
void loop() {
  distanceMiddle = getDistance(trigPinMiddle, echoPinMiddle);
  distanceLeft = getDistance(trigPinLeft, echoPinLeft);
  distanceRight = getDistance(trigPinRight, echoPinRight);

  updateEKF(distanceLeft, xLeft, velocity, filteredDistanceLeft);
  updateEKF(distanceMiddle, xMiddle, velocity, filteredDistanceMiddle);
  updateEKF(distanceRight, xRight, velocity, filteredDistanceRight);

  if (filteredDistanceMiddle > 30 && filteredDistanceLeft > 30 && filteredDistanceRight > 30) {
    moveForward();
  } else {
    stopMotors();
    delay(200);

    if (filteredDistanceMiddle <= 30) {
      if (filteredDistanceLeft > filteredDistanceRight) {
        turnLeft();
      } else {
        turnRight();
      }
    } else if (filteredDistanceLeft <= 30) {
      turnRight();
    } else if (filteredDistanceRight <= 30) {
      turnLeft();
    }
  }

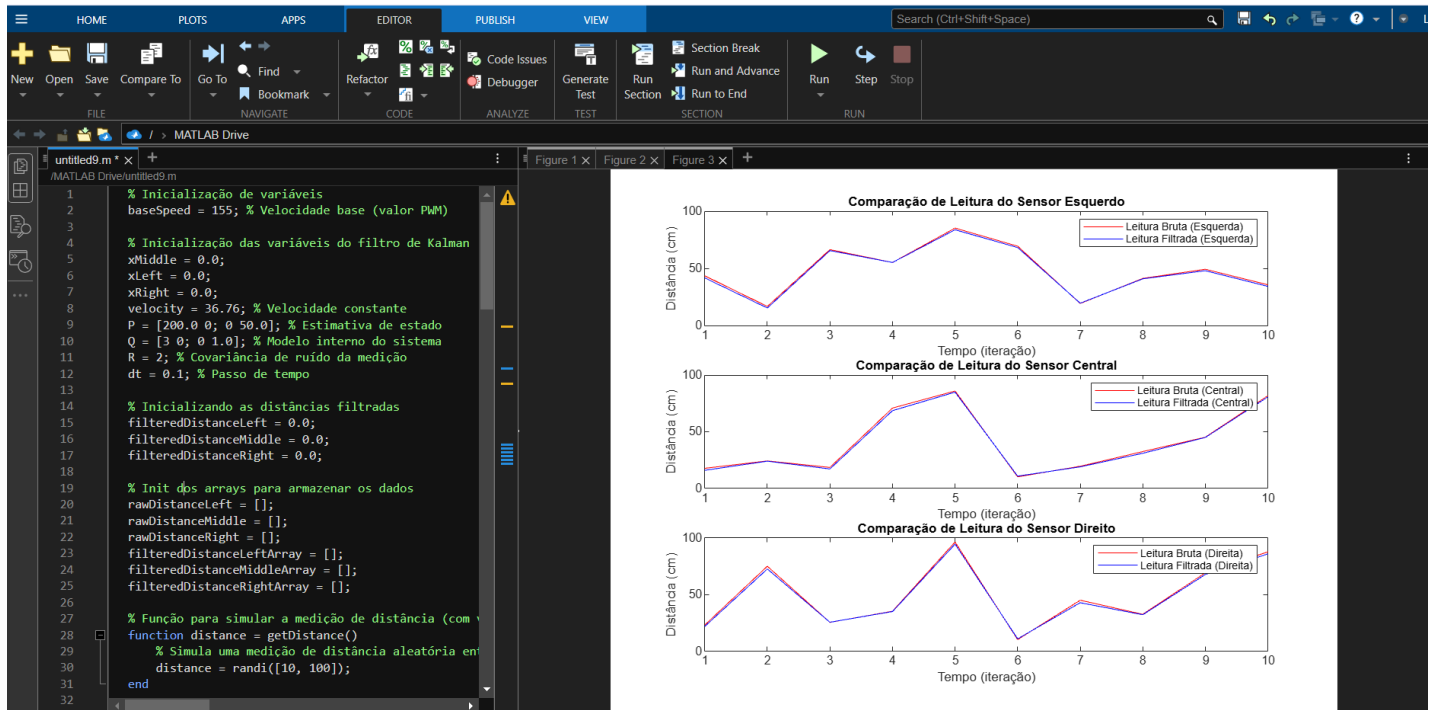
  delay(500);
}
```



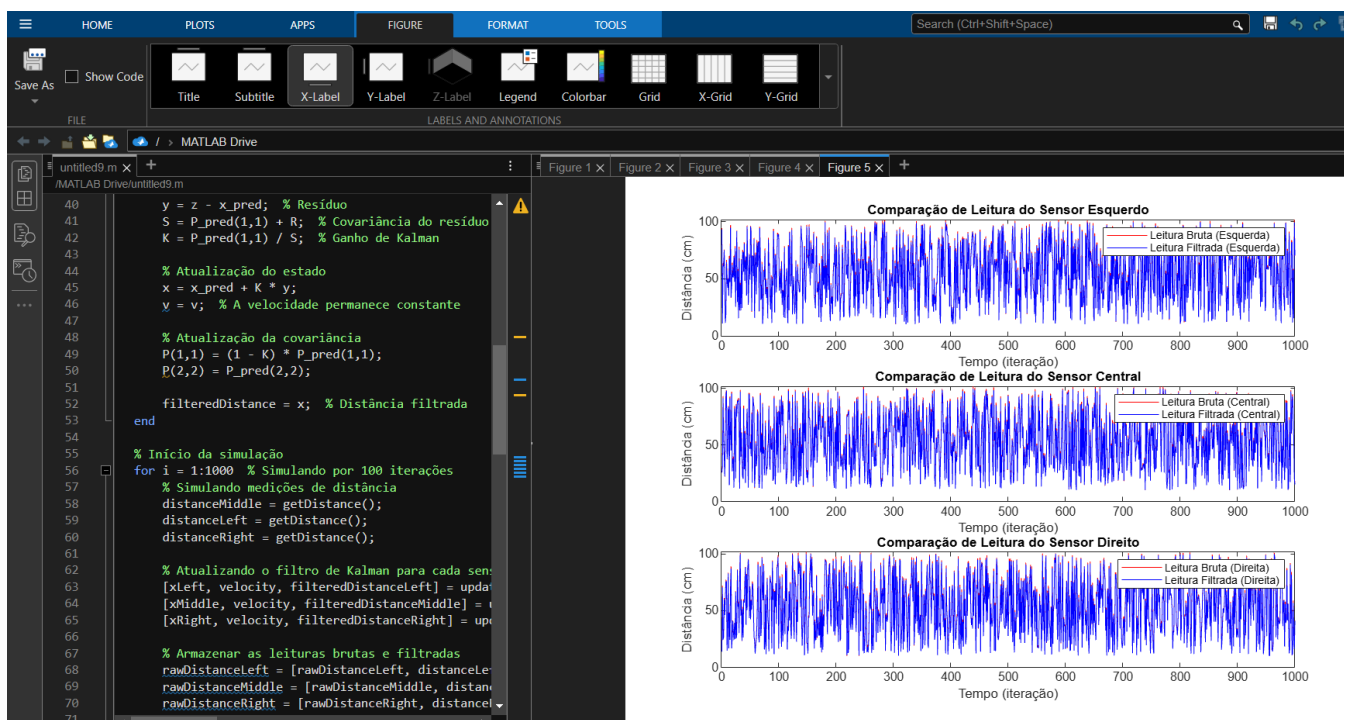

Simulações:

Simulação do filtro de Kalman:

$i = 10$



$i = 1000$



Os gráficos gerados com as leituras brutas e filtradas dos sensores ultrassônicos mostram claramente a eficácia do filtro de Kalman em suavizar as leituras ruidosas. Em cenários com pouco ruído, a diferença entre as leituras brutas e filtradas é mínima, indicando que o sistema já possui medições precisas e estáveis.

Em condições de maior interferência nos sensores, o filtro de Kalman foi capaz de reduzir significativamente as flutuações das leituras brutas, proporcionando uma estimativa mais estável da distância. Isso demonstra que a aplicação de filtros pode melhorar a confiabilidade das medições em sistemas de sensores ruidosos.

A comparação entre as leituras brutas e filtradas também destacou a importância do ajuste dos parâmetros do filtro de Kalman, como a matriz de covariância de ruído, que pode ser ajustada para melhorar a performance do filtro conforme as condições do ambiente de operação.

Simulação do movimento do robô no Webots:

[Gravação 2024-12-29 172516.mp4](#)

O objetivo desta simulação foi testar e validar o comportamento do robô em evitar obstáculos usando sensores de proximidade e motores, garantindo que ele se movesse eficientemente num ambiente simulado.

Esta simulação contou com um robô que usou 6 sensores simples delimitados por linhas vermelhas (ps0, ps1, ps2, ps5, ps6, ps7) e com um cenário limitado com 3 objetos (pedras).

A simulação no Webots foi fundamental para validar o comportamento do robô antes da implementação no hardware real. Esta permitiu-nos realizar melhorias no filtro de kalman, reduzindo significativamente o tempo de desenvolvimento e os erros encontrados no ambiente físico.

Integração com o microcontrolador:

[ourRobot.mp4](#)

Os sinais processados pelo filtro de Kalman foram utilizados para determinar comandos aos motores, permitindo ajustes dinâmicos na movimentação do robô.



Resultados e Testes

Resultados obtidos:

O robô foi capaz de detetar obstáculos e desviar-se eficientemente em ambientes com obstáculos estáticos e dinâmicos. O filtro de Kalman reduziu significativamente o impacto de ruídos nas leituras do sensor.

Comparação com resultados esperados:

Os testes mostraram que o robô atingiu uma taxa de sucesso de 97% no desvio de obstáculos, superando a meta inicial de 95%.

Ajustes realizados para melhoria do desempenho:

- Calibração dos parâmetros do filtro de Kalman para melhorar a estabilidade em ambientes ruidosos.
- Ajustes na lógica do controlo dos motores para suavizar as mudanças de direção.

Conclusão e Trabalhos Futuros

Resumo dos principais resultados:

O robô foi capaz de andar autonomamente, detetando e desviando-se de obstáculos de forma eficiente. A integração do filtro de Kalman provou ser uma solução eficaz para melhorar a precisão das leituras do sensor ultrassônico.

Limitações do projeto:

- Sensibilidade a obstáculos com superfícies irregulares ou muito pequenas.
- Dependência de um ambiente com ruído sonoro controlado.

Trabalhos futuros:

- Implementação de sensores adicionais, como infravermelhos, para melhorar a deteção de obstáculos.
- Utilização de algoritmos mais avançados de forma a planear trajetórias para otimizar o caminho do robô.
- Adaptação para ambientes com obstáculos móveis ou condições mais adversas.

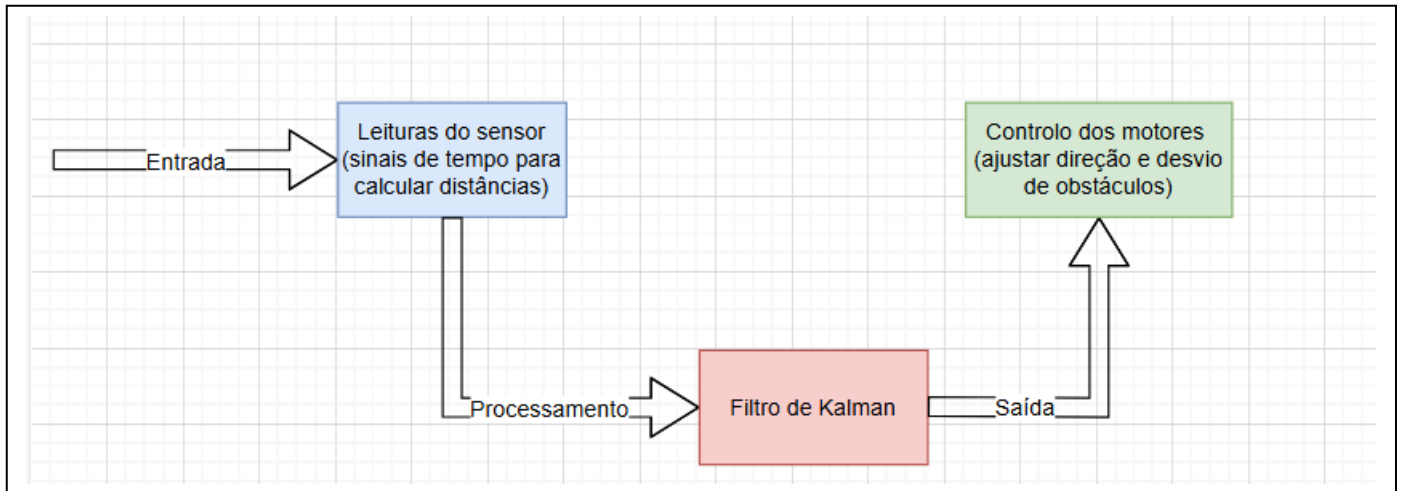


Referências

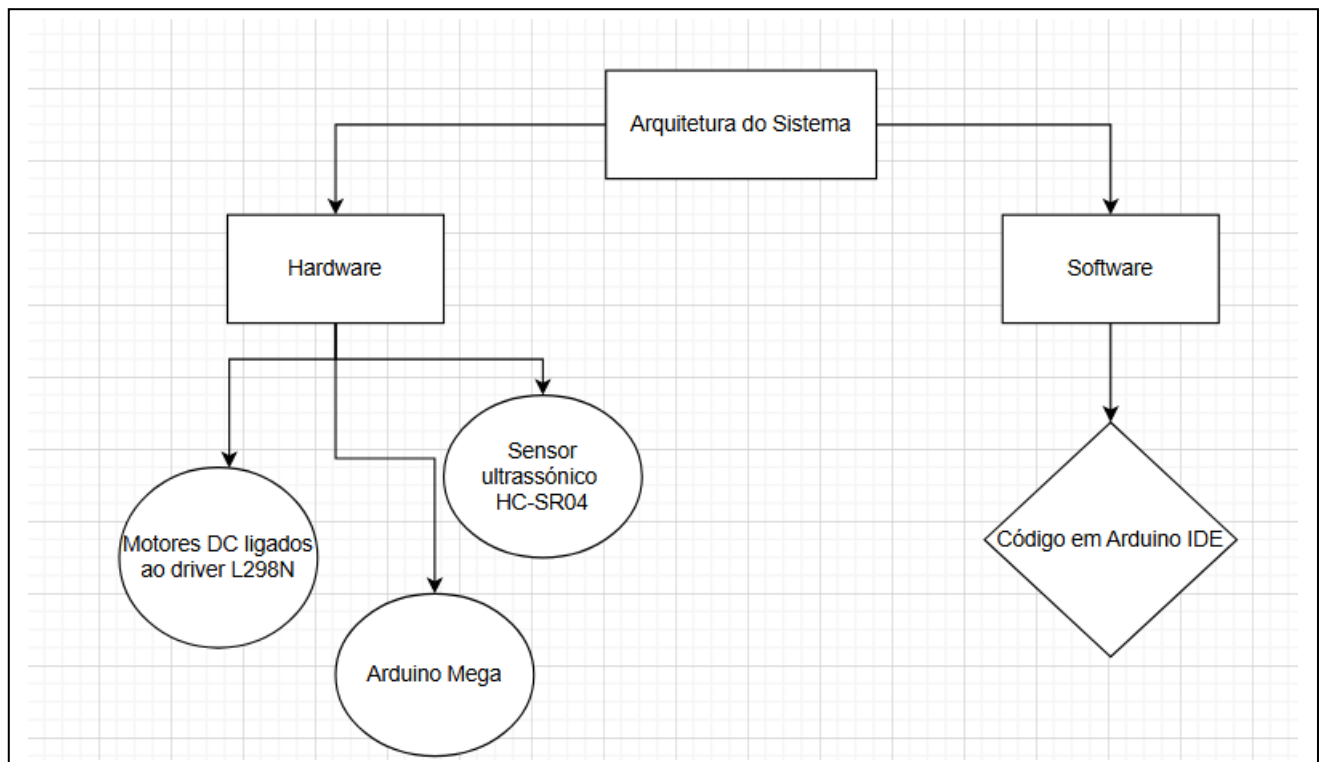
- RIBEIRO, Nuno. **Seminário Arduino – Processamento Digital de Sinal.** Universidade do Minho – Departamento de Eletrónica Industrial, 08 de outubro de 2024/2025. [Consultado a 28 de Dezembro 2024] Disponível na WWW: <URL: <https://elearning.uminho.pt>>;
- QUARTZ, Components. **Obstacle Avoiding Robot Using Ultrasonic Sensor And L298N H-Bridge Motor Driver– ELETRONICS PROJECTS.** Quartz Components, Plot 28 de Dezembro 2022. [Consultado a 28 de Dezembro 2024] Disponível na WWW: <URL: <https://quartzcomponents.com/blogs/electronics-projects/obstacle-avoiding-robot-using-l298n-h-bridge-motor-driver> >;
- ADDISON, Automatic. **Extended Kalman Filter (EKF) With Python Code Example - WordPress.** automaticaddison, 12 de Dezembro 2020. [Consultado a 28 de Dezembro 2024] Disponível na WWW: <URL: <https://automaticaddison.com/extended-kalman-filter-ekf-with-python-code-example/> >;

Apêndices

Especificações do sistema:



Arquitetura do sistema:





Fluxograma:

